

Heterogeneous Knowledge Distillation

Nabiev M.

Bakhteev O.

Abstract

We present a cross-architecture knowledge distillation method for transferring information between heterogeneous networks such as RNNs and CNNs. The key challenge is that their intermediate representations have different structure and depth, so a fixed, hand-crafted layer correspondence is often unreliable. Our approach learns a soft, input-dependent alignment between teacher and student layers, automatically deciding which internal components should match for each sample. Training augments the usual supervised objective with an additional term that encourages the student’s internal representations to become closer to the teacher’s under this learned alignment, while allowing different normalization choices depending on which model acts as the teacher.

1 Introduction

Contributions.

2 Problem statement

Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ be a labeled image dataset, where $\mathbf{x}_i \in \mathcal{X}$ and $y_i \in \mathcal{Y}$. We consider two architectures that solve the same task: an RNN-based network R and a CNN-based network C . In our method, one network plays the role of a *teacher* T and the other is the *student* S , where

$$T \in \{R, C\}, \quad S \in \{R, C\}, \quad T \neq S.$$

The student is trained using the standard supervised objective augmented with a cross-architecture representation alignment term.

2.1 Teacher and student representations

For an input $\mathbf{x}$, let the teacher expose a collection of intermediate representations

$$\mathbf{z}_1^T(\mathbf{x}), \dots, \mathbf{z}_{K_T}^T(\mathbf{x}),$$

and the student expose

$$\mathbf{z}_1^S(\mathbf{x}), \dots, \mathbf{z}_{K_S}^S(\mathbf{x}).$$

The two sets may have different cardinalities $K_T \neq K_S$ and different tensor structure.

In particular, when T (or S) is an RNN, we interpret $\mathbf{z}_k^{(\cdot)}(\mathbf{x})$ as a sequence/token representation $\mathbf{h}_k(\mathbf{x}) \in \mathbb{R}^{D_1}$. When T (or S) is a CNN, we interpret $\mathbf{z}_j^{(\cdot)}(\mathbf{x})$ as a feature map $\mathbf{A}_j(\mathbf{x}) \in \mathbb{R}^{D_2 \times H_j \times W_j}$. Our objective is to define a differentiable matching between teacher and student layers and encourage the student to mimic the teacher under this matching.

2.2 Aggregation into a shared space

Because the raw representations have different shape and semantics, we map each intermediate representation into a shared space of $d \times d$ positive semi-definite (PSD) matrices capturing second-order structure.

Vector (RNN-like) representations. Given a teacher or student representation that can be summarized as a vector, we first apply an aggregation/projection operator

$$\mathbf{u}_k(\mathbf{x}) := \text{aggr}_k(\mathbf{z}_1(\mathbf{x}), \dots, \mathbf{z}_k(\mathbf{x})) \in \mathbb{R}^d,$$

where $\text{aggr}_k(\cdot)$ takes a subset of tokens and may consist of pooling over them followed by a learnable linear projection. We then form a rank-1 PSD matrix

$$\mathbf{F}_k(\mathbf{x}) := \mathbf{u}_k(\mathbf{x}) \mathbf{u}_k(\mathbf{x})^\top \in \mathbb{R}^{d \times d}.$$

Spatial (CNN-like) representations. Given a feature map $\mathbf{A}_j(\mathbf{x}) \in \mathbb{R}^{D_2 \times H_j \times W_j}$, we first map channels to d dimensions using a learnable convolution:

$$\tilde{\mathbf{A}}_j(\mathbf{x}) := \text{Conv}(\mathbf{A}_j(\mathbf{x})) \in \mathbb{R}^{d \times H_j \times W_j}.$$

Let $P_j := H_j W_j$ and define the matrix of spatial features

$$\mathbf{B}_j(\mathbf{x}) := \text{Flatten}(\tilde{\mathbf{A}}_j(\mathbf{x})) \in \mathbb{R}^{d \times P_j}.$$

We aggregate spatial structure via the normalized Gram matrix

$$\mathbf{F}_j(\mathbf{x}) := \frac{1}{P_j} \mathbf{B}_j(\mathbf{x}) \mathbf{B}_j(\mathbf{x})^\top \in \mathbb{R}^{d \times d}.$$

Applying the appropriate construction to each intermediate representation yields:

$$\{\mathbf{F}_k^T(\mathbf{x})\}_{k=1}^{K_T} \subset \mathbb{R}^{d \times d}, \quad \{\mathbf{F}_j^S(\mathbf{x})\}_{j=1}^{K_S} \subset \mathbb{R}^{d \times d}.$$

2.3 Soft layer matching

We define the pairwise cost between teacher layer k and student layer j by the squared Frobenius distance:

$$\mathbf{C}(\mathbf{x}) \in \mathbb{R}^{K_T \times K_S}, \quad C_{kj}(\mathbf{x}) := \|\mathbf{F}_k^T(\mathbf{x}) - \mathbf{F}_j^S(\mathbf{x})\|_F^2.$$

We convert costs into a differentiable soft alignment matrix $\mathbf{\Gamma}(\mathbf{x}) \in \mathbb{R}^{K_T \times K_S}$ using a softmax over negative costs:

$$\mathbf{\Gamma}(\mathbf{x}) := \text{Softmax}(-\mathbf{C}(\mathbf{x})/\tau),$$

where $\tau > 0$ is an optional temperature.

Crucially, the normalization direction is chosen to reflect the distillation direction:

- **Teacher-to-student matching (row-wise):** $\sum_{j=1}^{K_S} \Gamma_{kj}(\mathbf{x}) = 1$ for each teacher layer k . Then each teacher representation is matched to a mixture of student representations.
- **Student-to-teacher matching (column-wise):** $\sum_{k=1}^{K_T} \Gamma_{kj}(\mathbf{x}) = 1$ for each student layer j . Then each student representation is matched to a mixture of teacher representations.

Both variants produce an input-dependent correspondence $\mathbf{\Gamma}(\mathbf{x})$, allowing different samples to induce different layer alignments.

2.4 Distillation objective

Let $\mathcal{L}_{\text{task}}(\mathbf{x}, y; \theta_S)$ denote the standard supervised loss of the student (e.g., cross-entropy), and let θ_S be the student parameters. The teacher parameters are fixed. We train the student by minimizing the expected objective

$$\min_{\theta_S} \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} \left[\mathcal{L}_{\text{task}}(\mathbf{x}, y; \theta_S) + \lambda \sum_{k=1}^{K_T} \sum_{j=1}^{K_S} \Gamma_{kj}(\mathbf{x}) C_{kj}(\mathbf{x}) \right],$$

where $\lambda \geq 0$ controls the strength of the distillation term.

The distillation regularizer encourages the student to produce intermediate representations whose second-order statistics match those of the teacher, while the soft alignment $\mathbf{\Gamma}(\mathbf{x})$ resolves the architectural mismatch by learning which teacher layers should correspond to which student layers.

3 Experiments

4 Conclusion.

References